AgileFlow AI

File 08 - Implementation Engineering Handbook

Human-in-the-Loop Multi-Agent Framework for Agile SDLC Automation

Document Purpose

• Provide implementation rules for the project foundation team and all later agent-development members.
• Create one engineering standard for frontend, backend, database, APIs, Git, documentation, testing, and AI-assisted coding.
• Prevent duplicate code, broken folder structures, inconsistent UI, missing functions, and merge conflicts.

1. Engineering Goal

The first implementation goal is to build a stable web application foundation for AgileFlow AI. The foundation must support project creation, workflow stages, approval gates, agent outputs, shared knowledge storage, traceability, and final reports before adding advanced AI automation.

First MVP Principle

• Build a working end-to-end workflow first: Client Input -> Requirements -> Design -> Tests -> Implementation -> Validation -> Final Report.
• Use mock or template outputs first where needed, then replace them with stronger AI-generated outputs later.
• A simple stable workflow is better than a complex broken system.

2. Recommended Technology Stack

Layer	Technology	Reason
Frontend	React + Vite + Tailwind CSS	Fast web UI development, reusable components, easy dashboard implementation.
Backend	Python FastAPI	Suitable for agent workflows, JSON APIs, async processing, and AI/tool integration.
Database	MongoDB	Flexible JSON-style storage for requirements, designs, tests, logs, reports, and traceability.
File Storage	Local storage in /storage for MVP	Simple storage for uploads, generated files, logs, and reports before cloud storage.
AI Layer	LLM prompts + structured JSON outputs first	Feasible for final-year prototype; no need to train a model from scratch.
Execution Layer	Docker sandbox later	Used for Agent implementation build/run validation after foundation is stable.

Layer	Technology	Reason
Version Control	GitHub with feature branches and PRs	Supports team collaboration, review, traceability, and safe integration.

3. Repository and Folder Structure

The repository must be created as a monorepo so frontend, backend, documentation, storage, and sample data are managed together.

```
agileflow-ai/
├── frontend/
├── backend/
├── docs/
│   ├── architecture/
│   ├── api-contracts/
│   ├── schemas/
│   ├── implementation-rules/
│   ├── ai-agent-rules/
│   └── meeting-notes/
├── storage/
│   ├── uploads/
│   ├── generated-artifacts/
│   ├── logs/
│   └── reports/
├── sample-data/
└── README.md
```

4. Backend Engineering Structure

The backend must follow a modular structure. Each major component owns its own router, service, repository, and schemas. Shared logic must stay in common modules.

```
backend/app/
├── main.py
├── core/
│   ├── config.py
│   └── constants.py
├── database/
│   └── mongodb.py
├── common/
│   ├── errors.py
│   ├── responses.py
│   ├── status.py
│   └── ids.py
├── modules/
│   ├── projects/
│   ├── requirements/
│   ├── design/
│   ├── testing/
│   ├── implementation/
│   ├── orchestrator/
│   ├── approvals/
│   ├── traceability/
│   └── reports/
└── schemas/
```

Backend Layer	Responsibility	Rule
Router	Defines API endpoints and validates request/response flow.	No business logic inside routers.
Service	Contains workflow logic, generation logic, validation, and orchestration calls.	Services call repositories, not the opposite.
Repository	Handles database read/write operations.	No UI/workflow logic inside repositories.
Schema	Defines request/response models and artifact JSON structures.	Reuse shared schemas; do not duplicate.
Common	Contains statuses, errors, IDs, and shared helpers.	Change only with group approval.

5. Frontend Engineering Structure

The frontend must be organized around shared layouts, reusable components, page modules, and API service files. Agent pages should not recreate components that already exist.

```
frontend/src/
├── app/
│   └── routes.jsx
├── assets/
├── components/
│   ├── common/
│   ├── layout/
│   ├── workflow/
│   ├── approvals/
│   ├── tables/
│   └── reports/
├── layouts/
│   └── MainLayout.jsx
├── pages/
│   ├── dashboard/
│   ├── projects/
│   ├── requirements/
│   ├── design/
│   ├── testing/
│   ├── implementation/
│   ├── orchestrator/
│   └── reports/
├── services/
├── styles/
│   ├── tokens.css
│   └── theme.js
└── utils/
```

6. Centralized Theme and Styling Rules

All styling must follow the centralized dark purple SaaS-style theme. Do not use random colors inside pages. Add reusable values to tokens.css or theme.js first.

Token	Value	Usage
--bg-main	#0B0714	Application background
--bg-sidebar	#120A24	Sidebar and navigation background

Token	Value	Usage
--bg-card	#1A102F	Cards, panels, modals
--purple-primary	#7C3AED	Primary actions and active states
--purple-accent	#A855F7	Highlights and secondary accent
--text-primary	#F8FAFC	Main text on dark surfaces
--text-secondary	#CBD5E1	Muted text and descriptions
--success	#22C55E	Approved, passed, completed
--warning	#F59E0B	Pending, waiting, warning
--error	#EF4444	Rejected, failed, vulnerability
--info	#38BDF8	In progress, informational status

- Use Tailwind utility classes with design tokens where possible.
- Do not hardcode colors repeatedly inside individual pages.
- All pages must use the same sidebar, topbar, page header, card style, status badge, and table style.
- Use one typography scale for headings, body text, labels, and captions.
- Every screen must include loading, error, and empty states where data is fetched.

7. Reusable Component Rules

Reusable components must be created before building full pages. Each component should have a single responsibility and clear props.

Component	Purpose	Used In
MainLayout	Sidebar, topbar, content area wrapper.	All authenticated pages
PageHeader	Title, subtitle, action button area.	Every main page
StatusBadge	Shows PENDING, APPROVED, FAILED, COMPLETED, etc.	Workflow, tables, reports
WorkflowStepper	Shows current SDLC stage and approval gates.	Dashboard, project workflow, orchestrator
ApprovalGateCard	Approve, reject, request changes with comment.	H1 to H5 approval screens
AgentOutputPanel	Displays generated artifact summary.	All agent pages
DataTable	Reusable table with search/filter-ready layout.	Backlogs, tests, traceability, reports
LogViewer	Displays build, runtime, test, and security logs.	Implementation and validation pages
TraceabilityTable	Shows links from requirement to design, test, code, and report.	Traceability report
EmptyState/ErrorState/LoadingState	Consistent state feedback.	All data-driven pages

Reusable Component Rule

• If the same UI appears in more than one page, create it as a reusable component.
• Do not create separate status badges, buttons, tables, or approval cards inside each agent folder.

• Shared UI changes must be reviewed because they affect all members.

8. API and Service Integration Rules

Frontend pages must not call fetch or axios directly inside page components. Use service files to keep API calls consistent.

```
frontend/src/services/
├── apiClient.js
├── projectService.js
├── requirementService.js
├── designService.js
├── testingService.js
├── implementationService.js
├── orchestratorService.js
├── approvalService.js
└── reportService.js
```

Rule	Explanation
Use one API client	Base URL, headers, and error handling must be centralized.
Use service files	Pages call service methods, not raw endpoints.
Keep response format consistent	All APIs should return success, message, data, and errors where needed.
Update API_CONTRACT.md	Any endpoint change must be documented before or with the code change.
Handle all UI states	Each API-driven page must handle loading, empty, error, and success states.

9. Common Status and Approval Rules

All agents and UI screens must use the same workflow statuses and approval gates. Do not create new status names without approval.

Common Status	Meaning
PENDING	Task exists but has not started.
IN_PROGRESS	Agent or workflow stage is running.
WAITING_FOR_APPROVAL	Output is ready and waiting for human decision.
APPROVED	Human reviewer approved the output.
REJECTED	Human reviewer rejected the output.
FAILED	Build, test, security, or workflow step failed.
COMPLETED	Stage completed successfully.

Gate	Approval Area	Owner
H1	Requirement approval	Requirement Gathering and Analysis Agent
H2	Design and planning approval	Project Management and Design Agent
H3	Pre-code test approval	Testing, Validation and Security Agent
H4	Implementation approval	Implementation and Git Automation Agent
H5	Final validation approval	Testing, Validation and Security Agent

10. Orchestrator Integration Rules

The Orchestrator controls workflow transitions, approval gates, handovers, failure routing, retry limits, and state management. Individual agents should not directly skip stages.

```
PROJECT_CREATED
→ REQUIREMENTS_IN_PROGRESS
→ H1_WAITING_APPROVAL
→ DESIGN_IN_PROGRESS
→ H2_WAITING_APPROVAL
→ TEST_PLANNING_IN_PROGRESS
→ H3_WAITING_APPROVAL
→ IMPLEMENTATION_IN_PROGRESS
→ H4_WAITING_APPROVAL
→ VALIDATION_IN_PROGRESS
→ H5_WAITING_APPROVAL
→ COMPLETED
```

- Each agent must save its output to the Shared Knowledge Base before requesting approval.
- Each approval action must create a workflow event.
- Each failure must be routed through the Orchestrator, not manually hidden inside an agent page.
- The retry count must be tracked for build failures, test failures, and security validation failures.
- The final report must only be generated after H5 approval.

11. Shared Knowledge Base Rules

The Shared Knowledge Base is the common storage layer for artifacts generated by all agents. Every important artifact must be stored with project_id, artifact_id, status, version, created_by_agent, and traceability links where needed.

Collection	Stores
projects	Project metadata, domain, current workflow status, owner, dates
requirements	SRS summary, requirements, user stories, acceptance criteria, REQ IDs
design_artifacts	Backlog, sprint plan, UML data, API plan, DB schema, UI flow, wireframe plan
test_cases	Pre-code test cases, executable test skeleton metadata, security test cases
implementation_artifacts	Generated modules, file tree, repo metadata, code mapping
execution_logs	Build logs, runtime logs, Docker logs, agent logs
test_results	Unit/API/UI/regression results, pass/fail data, coverage summary
bug_reports	Failed tests, bug IDs, severity, linked requirement/test/code

Collection	Stores
security_findings	Security issue, severity, location, suggested fix, status
traceability_links	Requirement -> design -> test -> code -> bug/security -> report links
workflow_events	Timeline of actions, approvals, failures, retries, handovers

12. Git, Branching, and Pull Request Rules

The develop branch must always remain runnable. Incomplete work must stay inside feature branches until reviewed.

```
main                        # Stable final version only
develop                     # Integration branch
feature/project-foundation   # Initial setup and shared foundation
feature/ui-design-system     # Frontend theme and reusable components
feature/agent-requirements   # Requirement agent
feature/agent-design-planning# Design and planning agent
feature/agent-implementation # Implementation and Git automation agent
feature/agent-testing-validation # Testing and validation agent
feature/orchestrator         # Workflow control and approvals
feature/reports-traceability # Final reports and traceability
```

Rule	Description
No direct push to main	main is used only for stable milestone releases.
PR required for develop	All feature branches must be merged through pull request review.
Pull before work	Always pull latest develop before starting daily work.
Small commits	Commit logical units of work, not huge mixed changes.
Explain AI changes	PR description must mention if AI agent generated or modified code.
Run before PR	Frontend and backend must run locally before opening PR.

13. AI Agent Development Rules

Critical Rule for AI Coding Tools

- AI agents can help with coding, but every generated change must be reviewed by the responsible member.
- The prompt must clearly mention allowed folders, do-not-modify files, API contract, data schemas, and expected output.
- Never allow an AI agent to edit the whole repository without boundaries.

```
AI Prompt Template


You are working on AgileFlow AI.
Component: [component name]
Current branch: [branch name]
Allowed folders: [list]
Do not modify: [list]
Follow docs: README, API_CONTRACT, DATA_SCHEMAS,
```

```
AI_AGENT_RULES, Implementation Engineering Handbook.
Task: [specific task]
Requirements:
- Use existing shared models.
- Do not create duplicate APIs or schemas.
- Do not rename existing routes or fields.
- Keep code simple and readable.
- Return changed files and testing steps.
```

14. Protected Files and Change Control

The following files affect the whole system. They should be changed only by the group leader or after team approval.

```
Protected shared files:
backend/app/main.py
backend/app/core/config.py
backend/app/database/mongodb.py
backend/app/common/status.py
backend/app/common/responses.py
frontend/src/app/routes.jsx
frontend/src/layouts/MainLayout.jsx
frontend/src/styles/tokens.css
frontend/src/styles/theme.js
docs/api-contracts/API_CONTRACT.md
docs/schemas/DATA_SCHEMAS.md
```

15. Code Quality and Naming Standards

Area	Standard
Files and folders	Use lowercase kebab-case or clear module names. Keep names meaningful.
Python functions	Use snake_case and clear service/repository names.
React components	Use PascalCase component names.
API routes	Use plural resource names and project-scoped routes.
IDs	Use REQ-001, DES-001, TEST-001, CODE-001, BUG-001, SEC-001, TRACE-001.
Errors	Return structured errors with message and details.
Comments	Add comments for workflow decisions, not obvious code.
Secrets	Never commit API keys, tokens, passwords, or connection strings.

16. Testing and Local Run Checklist

Before opening a pull request, each member must complete a basic local verification checklist.

☐ Frontend starts without console errors.

☐ Backend starts without import or dependency errors.

☐ MongoDB connection works or mock storage fallback is documented.

☐ New API endpoints return expected response shape.

☐ New UI pages show loading, empty, error, and success states.

☐ Approval buttons update status correctly where implemented.

☐ No unrelated files were modified.

☐ Documentation updated if API, schema, route, or workflow changed.

17. Security and Data Handling Rules

- Use .env files for local configuration and never commit secrets.
- Add .env.example with placeholder values only.
- Sanitize file names for uploads and store uploaded files under project-specific folders.
- Do not store real client confidential data in sample-data.
- Do not expose stack traces to frontend users; use structured error messages.
- All final demo screenshots must hide secrets, tokens, connection strings, and private data.
- Security findings must be stored and linked to the affected requirement, test, or code artifact where possible.

18. Documentation Update Rules

Documentation must be updated together with implementation changes. This is important because new chats, AI agents, and team members will use these documents as the knowledge base.

When This Changes	Update This Document
API endpoint	API_CONTRACT.md
Data field or schema	DATA_SCHEMAS.md
UI page or navigation	UI_PAGES.md / Frontend Implementation Guide
Reusable component	Reusable Component Plan
Workflow stage or approval logic	Orchestrator Workflow and Approval Rules
Branch ownership or task responsibility	Task Assignment and Branching Rules
Setup or dependency	README.md and Implementation Engineering Handbook if major

19. Definition of Done

A task is done only when:

• Code is implemented inside the correct assigned folder.
• The feature follows API contracts, schema rules, theme rules, and workflow rules.
• Frontend and backend run locally without breaking existing features.
• The implementation creates or updates needed workflow events, statuses, and traceability links where applicable.
• The PR includes changed files, testing steps, screenshots for UI changes, and known limitations.
• Documentation is updated if routes, schemas, workflows, or reusable components changed.

20. First Foundation Sprint Checklist

The implementation environment starting team should complete the following tasks before all members start heavy component development.

1. Create GitHub repository and branches: main, develop, feature/project-foundation, feature/ui-design-system.
2. Create monorepo folder structure: frontend, backend, docs, storage, sample-data.
3. Create README, CONTRIBUTING, API_CONTRACT, DATA_SCHEMAS, AI_AGENT_RULES, and this engineering handbook.

4. Set up React + Vite + Tailwind frontend foundation.
5. Set up FastAPI backend foundation with modular folder structure.
6. Create MongoDB connection with .env.example.
7. Create project model and basic project APIs.
8. Create workflow status model and approval model.
9. Create basic Orchestrator next-step logic.
10. Create shared theme tokens and reusable layout components.
11. Create dashboard and project workflow page skeletons.
12. Merge only after the foundation runs successfully on both frontend and backend.

21. Final Leader Reminder

Group Leader Control Point

• Do not allow members to start separate random implementations before the foundation is merged into develop.
• Keep develop runnable at all times.
• Use the documents as the source of truth for AI agents and team members.
• Integrate one component at a time: Requirement -> Design -> Pre-Code Testing -> Implementation -> Post-Code Validation -> Reports.
• Incomplete feature branches are acceptable. A broken develop branch is not acceptable.